Below is a **complete Java version-wise feature list** (Java SE **1.0 → Java 21 (LTS)** and beyond if applicable) in a **clean table format**, including **major language features, JVM changes, APIs, and developer tools**.

✅ Note: Some "versions" are **Java SE**, some are **JDK releases**. I'm listing them in the most accepted public Java release order.

✅ Java Version-wise Features (Java 1.0 → Latest)

Java Version	Release Year	Major Features Introduced (Detailed)
Java 1.0	1996	First stable Java release, **JVM + Bytecode**, **Applet support**, **AWT GUI**, **Basic networking**, **I/O streams**, **Threading**, **Exception handling**
Java 1.1	1997	**Inner classes**, **JavaBeans**, **JDBC**, **RMI**, **Reflection**, **Event delegation model**, **Object serialization**, **Internationalization (i18n)**
Java 1.2 (Java 2)	1998	Big release: **Swing GUI**, **Collections Framework (List/Set/Map)**, **JIT compiler improvements**, **Security model (policy-based)**, **Access Controller**, **Java Plug-in**, **StrictMath**
Java 1.3	2000	**HotSpot JVM default**, **JavaSound**, **RMI over IIOP**, **JNDI improvements**, **Timer API**, **JNI enhancements**, performance improvements
Java 1.4	2002	**assert keyword**, **NIO (non-blocking I/O)**, **Regex API**, **Logging API**, **XML processing (JAXP)**, **Exception chaining**, **IPv6 support**, **Image I/O**
Java 5 (1.5)	2004	Huge language change: **Generics**, **Annotations**, **Enums**, **Autoboxing/Unboxing**, **Varargs**, **Enhanced for-each loop**, **Static imports**, **Concurrent package (java.util.concurrent)**, **Formatter**, **Metadata**
Java 6	2006	**Scripting API (JSR-223)**, **Compiler API**, **JDBC 4.0**, **Pluggable annotations**, **Web services improvements**, **Performance/HotSpot upgrades**, **Console API**, **JAX-WS/JAXB included**
Java 7	2011	**Project Coin**: *try-with-resources*, *multi-catch*, *diamond operator*, *strings in switch*, *binary literals*, *underscores in numeric literals*; **NIO.2**, **Fork/Join Framework**, **invokedynamic**, better GC
Java 8	2014	Biggest modern Java: **Lambda expressions**, **Functional interfaces**, **Streams API**, **Default & static methods in interfaces**, **Optional**, **New Date-Time API (java.time)**, **Nashorn JS engine**, **CompletableFuture**, **Base64 API**

Java 9	2017	Major architecture: **JPMS Modules (Project Jigsaw)**, **JShell REPL**, **Process API improvements**, **HTTP/2 client (incubator)**, **Collection factory methods (List.of)**, **Private interface methods**, **Multi-release JARs**
Java 10	2018	**Local variable type inference (var)**, **Application CDS**, **Parallel full GC for G1**, **Root certificates included**, container improvements
Java 11 (LTS)	2018	**HTTP Client API finalized**, **String methods** (isBlank, lines, strip), **Files methods**, **var in lambda parameters**, **Nest-based access control**, **Flight Recorder**, removal of Java EE modules from JDK
Java 12	2019	**Switch expressions (preview)**, **JVM constants API**, **Shenandoah GC (experimental)**, improvements in G1
Java 13	2019	**Text Blocks (preview)**, **Switch expressions 2nd preview**, **Dynamic CDS archives**, **Reimplement legacy Socket API**
Java 14	2020	**Switch expressions finalized**, **Records (preview)**, **Pattern matching for instanceof (preview)**, **Helpful NullPointerExceptions**, **Text Blocks (2nd preview)**
Java 15	2020	**Text Blocks finalized**, **Sealed classes (preview)**, **Hidden classes**, **Records (2nd preview)**, **ZGC production-ready**, Nashorn removed
Java 16	2021	**Records finalized**, **Pattern matching instanceof finalized**, **Sealed classes (2nd preview)**, **Vector API (incubator)**, **Foreign linker API (incubator)**
Java 17 (LTS)	2021	**Sealed classes finalized**, **Enhanced PRNG**, **Pattern matching switch (preview)**, **Foreign Function & Memory API (incubator)**, **Strong encapsulation**, **Remove RMI Activation**
Java 18	2022	**Simple web server**, **UTF-8 default**, **Code snippets in Javadoc**, **Pattern matching switch (2nd preview)**, **Vector API updates**
Java 19	2022	**Virtual Threads (preview - Project Loom)**, **Structured concurrency (incubator)**, **Record patterns (preview)**, **Pattern matching switch (preview)**, **Foreign Function & Memory API (preview)**
Java 20	2023	**Virtual Threads (2nd preview)**, **Record patterns (2nd preview)**, **Pattern matching switch (4th preview)**, **Structured concurrency (2nd incubator)**, **Scoped values (incubator)**
Java 21 (LTS)	2023	✅Major LTS: **Virtual Threads finalized**, **Pattern matching switch finalized**, **Record patterns finalized**, **Sequenced Collections**, **String Templates (preview)**, **Unnamed patterns & variables (preview)**, **Scoped values (preview)**, **Foreign Function & Memory API**

		finalized, **Generational ZGC**
Java 22	2024	**String Templates (2nd preview)**, **Unnamed variables/patterns (2nd preview)**, **Stream Gatherers (preview)**, **Class-File API (preview)**, **Foreign Function & Memory updates**, performance + GC improvements
Java 23	2024	New previews/incubators expected (based on Java release cadence): further evolution of **Stream Gatherers**, **Class-File API**, and **String Templates**; performance/GC improvements *(exact finalized list depends on official GA)*

Java Feature Evolution by Category (Quick Summary Table)

Category	Key Versions	Examples of Features
Core Language Enhancements	5, 7, 8, 10, 14, 16, 17, 21	Generics, lambdas, var, records, sealed classes
Concurrency	5, 7, 8, 19→21	Executor framework, ForkJoin, CompletableFuture, Virtual Threads
Collections	1.2, 8, 9, 21	Collections framework, Streams, immutable factory methods, Sequenced collections
JVM/Architecture	9, 11, 17, 21	Modules, encapsulation, improved GC, Panama API
Developer Tools	9, 11, 18	JShell, Flight Recorder, simple web server
Pattern Matching	14→21	instanceof, switch, record patterns
Strings Improvements	11, 13→15, 21+	strip/isBlank, text blocks, templates

---

✅ **If you want, I can also give:**

1. **Java version → feature → small code snippet table** (interview ready)
2. **Java feature roadmap** (what to learn first and why)
3. A printable **PDF cheat sheet (Java 1.0 → 21)**

Absolutely ✅
Here is a **Java Feature Roadmap (what to learn first and why)** — designed like an **industry + interview + real project** learning path.

I'm organizing it into **phases**, and in every phase I'll tell you:

- ✅ **What to learn**
- ✅ **Why it matters**
- ✅ **Where it is used in real projects**
- ✅ **Which Java version introduced it**

✅ Java Feature Roadmap (What to Learn First & Why)

PHASE 0 — Foundation Setup (Before Features)

✅ Learn first

- JDK installation, JAVA_HOME, PATH
- IDE: IntelliJ / Eclipse
- Java compilation flow:
  - .java → javac → .class → JVM
- JAR files, classpath, packages

✅ Why first?

Because every Java feature is useless without knowing:

- compilation
- project structure
- class loading

PHASE 1 — Core Java (Must Learn First)

Goal: Strong base for any Java project

Priority	Feature	Version	Why Learn First?	Real Use
1	Variables, Data types, Operators	1.0	Everything depends on this	all Java code
2	if/else, loops, break/continue	1.0	Core logic building	validation, processing
3	Methods, parameters, return types	1.0	Code modularity	utility methods
4	Arrays	1.0	Base DS for performance	caching, batch processing

5	OOP: Class/Object	1.0	Java is OOP-centered	Spring, Android
6	Constructor	1.0	Object creation control	model classes
7	this, static, final	1.0	memory + design clarity	constants, services
8	Inheritance	1.0	reuse + polymorphism	framework design
9	Abstraction (abstract class)	1.0	architecture + templates	service base classes
10	Interface	1.0	standardization + flexibility	DI, adapters

✅ **Outcome:** You can write **any basic Java program correctly**.

PHASE 2 — Object-Oriented Mastery (Real Java Skill)

Goal: Java developer thinking (not just syntax)

Priority	Feature	Version	Why	Real Use
1	Encapsulation, getters/setters	1.0	security of object data	DTO, entity
2	Polymorphism (runtime)	1.0	key to frameworks	Spring injection
3	Method overriding/overloading	1.0	backbone of OOP	API design
4	Inner classes	1.1	used in callbacks & patterns	UI handlers
5	Enum	5	strongly typed constants	order status, roles
6	Annotations	5	modern Java depends on it	Spring, Hibernate
7	Reflection basics	1.1	frameworks work using it	dependency injection

✅ **Outcome:** You understand how Spring/Hibernate work internally.

PHASE 3 — Exceptions + File Handling (Must for Production)

Goal: Write safe and stable code

Priority	Feature	Version	Why	Real Use
1	try-catch-finally	1.0	error handling	runtime safety
2	throw / throws	1.0	controlled failures	service layer
3	Custom exceptions	1.0	domain-specific errors	Banking exceptions
4	try-with-resources	7	auto closing resources	file/db streams
5	I/O Streams	1.0	real world data handling	file processing
6	NIO / NIO.2	1.4 / 7	faster + modern file APIs	high performance

✅ **Outcome:** You can build stable backend services.

PHASE 4 — Collections + Generics (Most Interview Important)

Goal: Master the #1 topic in interviews & real work

Priority	Feature	Version	Why	Real Use
1	Collections (List/Set/Map)	1.2	used in every project	APIs, DB results
2	ArrayList vs LinkedList	1.2	performance	real systems
3	HashMap vs TreeMap	1.2	speed vs sorted	caching, ranking
4	HashSet vs TreeSet	1.2	uniqueness	Filtering
5	Comparable vs Comparator	1.2	sorting logic	leaderboards

6	Generics	5	type safety	framework APIs

✅ **Outcome:** You become a serious Java developer.

PHASE 5 — Multithreading + Concurrency (Advanced but Required)

Goal: Write scalable, fast applications

Priority	Feature	Version	Why	Real Use
1	Thread class & Runnable	1.0	concurrency base	background tasks
2	synchronized, locks	1.0	prevent data race	shared state
3	ExecutorService	5	modern threading	scalable systems
4	ForkJoin Framework	7	parallel computation	batch pipelines
5	CompletableFuture	8	async workflows	API orchestration
6	Virtual Threads	21	huge scalability	Microservices

✅**Outcome:** You can handle high-load services (enterprise-grade).

PHASE 6 — Java 8 Revolution (Must Learn After Core)

Goal: Become modern Java developer

Priority	Feature	Version	Why	Real Use
1	Lambda Expressions	8	clean functional style	filtering data
2	Functional interfaces	8	used everywhere	Supplier/Consumer
3	Streams API	8	powerful data processing	collections pipeline
4	Optional	8	avoids NullPointer	null safety
5	Method reference	8	readable code	stream ops

6	Default methods in interface	8	backward compatible APIs	library design
7	New Date-Time API (java.time)	8	correct time handling	enterprise systems

✅ **Outcome:** You can work in 95% Java jobs confidently.

PHASE 7 — JVM + Project Structure (Intermediate to Advanced)

Goal: Become backend / platform-level Java engineer

Priority	Feature	Version	Why	Real Use
1	JVM Memory (Heap/Stack)	—	debugging + performance	tuning servers
2	Garbage Collection basics	—	prevents crashes	high load systems
3	Modules (JPMS)	9	large apps & security	enterprise modular apps
4	ClassLoader	—	frameworks depend on it	plugin architectures
5	JAR / WAR structure	—	deployment knowledge	Spring Boot

✅ **Outcome:** You can debug real production issues.

PHASE 8 — Modern Java Features (Java 11 → 21)

Goal: Write latest clean modern code

Priority	Feature	Version	Why	Real Use
1	HTTP Client API	11	REST calls	API integration
2	var (local inference)	10	modern readability	clean code
3	Text Blocks	15	JSON/SQL	REST + queries

			readability	
4	Records	16	immutable DTOs	APIs + microservices
5	Sealed Classes	17	controlled inheritance	domain modeling
6	Pattern matching instanceof	16	cleaner type checks	parsing logic
7	Pattern matching switch	21	modern switch	rule engines
8	Sequenced Collections	21	better collection semantics	ordered APIs
9	Foreign Function API (Panama)	21	native integration	performance apps

✅ **Outcome:** Your code looks like senior engineer code.

□ The Perfect Learning Order (Checklist)

If you want **exact order**, follow this:

✅ Beginner (0 → 30%)

1. Core syntax
2. OOP
3. Arrays + Strings
4. Exception handling
5. File I/O

✅ Intermediate (30 → 70%)

6. Collections + Generics
7. Java 8 (Lambda, Streams)
8. Multithreading (Executor, CompletableFuture)

✅ Advanced (70 → 100%)

9. JVM internals (GC, memory)
10. Modules (Java 9)
11. Modern Java (Records, Sealed, Pattern matching)
12. Virtual Threads (Java 21)

□ Roadmap Based on Career Goals

✅ If your goal is Spring Boot / Backend Developer

Focus most on:

- Collections + Generics
- Streams API
- Annotations
- Exceptions
- Multithreading
- JVM basics

✅ If your goal is Android Developer

Focus most on:

- OOP + Interfaces
- Collections
- Threads
- File I/O
- Lambdas

✅ If your goal is FAANG Interviews

Focus most on:

- Collections (deep)
- time complexity
- streams
- concurrency
- design patterns

If you want, I can also provide:

✅ **Java roadmap with weekly plan (8 weeks / 12 weeks / 24 weeks)**
✅ Version-wise **Java code examples for each feature**
✅ A printable **one-page roadmap chart (PDF)**

Sure ✅
Here is the **MASTER TABLE of Java Features (Version-wise)** — this is the **single most complete interview + project-ready table**.

✅ **Java Master Table (Version → All Major Features)**

Java Version	Year	Language Features	JVM / Performance	APIs / Libraries / Tools
Java 1.0	1996	Classes, Objects, Inheritance, Interfaces, Exceptions, Threads	JVM + Bytecode	Basic I/O (java.io), Networking (java.net), AWT, Applets
Java 1.1	1997	Inner Classes	Improved JVM	JDBC, RMI, Reflection, Serialization, JavaBeans, Event delegation
Java 1.2 (Java 2)	1998	—	Better JIT	Collections Framework, Swing, Security model improvements
Java 1.3	2000	—	HotSpot becomes standard	JavaSound, RMI over IIOP, JNDI improvements
Java 1.4	2002	assert	Performance improvements	Regex, Logging API, NIO, XML processing (JAXP), IPv6
Java 5	2004	Generics, Annotations, Enums, Varargs, Enhanced for-loop, Autoboxing, Static import	—	java.util.concurrent, Formatter, Metadata improvements
Java 6	2006	—	Performance, GC	Scripting API (JSR-223), Compiler API, JDBC 4.0, JAX-WS/JAXB
Java 7	2011	try-with-resources, multi-catch, diamond <>, strings in switch, binary literals, underscores in numbers	invokedynamic	NIO.2, Fork/Join framework
Java 8	2014	Lambda, Functional interfaces, Default methods	—	Streams, Optional, java.time, CompletableFuture, Base64

Java 9	2017	Private methods in interface	Modules (JPMS)	JShell, HTTP Client (incubator), List.of, Multi-release JARs
Java 10	2018	var	Better GC	Application CDS, container improvements
Java 11 (LTS)	2018	var in lambda params	Better runtime	HTTP Client finalized, Flight Recorder, String/File enhancements
Java 12	2019	Switch expressions (preview)	Shenandoah GC (experimental)	JVM constants API
Java 13	2019	Text Blocks (preview)	—	Socket API reimplementation
Java 14	2020	Switch expressions finalized, Records (preview), instanceof pattern (preview)	Helpful NPEs	—
Java 15	2020	Text Blocks finalized, Sealed classes (preview)	Hidden classes, ZGC stable	Nashorn removed
Java 16	2021	Records finalized, instanceof pattern finalized	Vector API incubator	Foreign Linker incubator
Java 17 (LTS)	2021	Sealed classes finalized	Strong encapsulation	Pattern matching switch (preview), PRNG improvements
Java 18	2022	—	UTF-8 by default	Simple web server, Javadoc code snippets
Java 19	2022	Record patterns (preview), switch pattern matching (preview)	Virtual threads (preview)	Foreign Function & Memory API (preview), Structured concurrency (incubator)
Java 20	2023	Record patterns (2nd preview)	Virtual threads (2nd preview)	Scoped values (incubator)
Java 21 (LTS)	2023	Record patterns finalized, switch pattern matching finalized	Virtual threads finalized, Generational ZGC	Sequenced collections, Foreign Function & Memory API finalized

Java 22	2024	String templates (preview), unnamed vars/patterns (preview)	Performance improvements	Stream gatherers (preview), Class-file API (preview)
Java 23	2024	More previews refined	—	Continuation improvements (depends on GA list)

If you want, I can also give you a **"Master Table – Interview Edition"** with columns like:

✅ Feature → ✅ Version → ✅ Code Example → ✅ Where Used in Real Projects → ✅ Interview Questions